A0 Cheat Sheet

Your team's code (`main` of 19 Oct, `f78faff3`) • the API you need for the interview • read it once, keep it next to CLion

team osw26e3 • SWEB x86-64

1 How to get to everything

```
UserProcess* process = currentThread->getProcess();
ArchMemory& mem    = process->getAddressSpace(); // page tables
Mutex& lock        = process->getThreadLock();   // per-process lock
Loader& loader     = process->getLoader();       // ELF loader
UserThread* ut     = (UserThread*) currentThread; // UserThread fields
```

- `currentThread` = the thread running right now (global `Thread*`).
- Pointer → `->`, reference → `.`: `process->getAddressSpace().mapPage(...)`

× `currentThread->loader_`, `(UserProcess*) currentThread` = the tutorials' (upstream) style - **not** your team's code.

2 Pages: constants & formulas

`PAGE_SIZE`	4096	one page = one frame
`USER_BREAK`	`0x800000000000ULL`	below = user, above = kernel
`USER_BREAK_VPN`	`USER_BREAK/PAGE_SIZE`	same, as page number

```
size_t vpn    = address / PAGE_SIZE;   // address -> page number
size_t addr   = vpn * PAGE_SIZE;       // page number -> address
size_t offset = address % PAGE_SIZE;   // position inside the page
bool aligned  = (address % PAGE_SIZE == 0);
// next page: vpn + 1   (NOT vpn + PAGE_SIZE!)
```

vpn = virtual page (what the program sees) **ppn** = physical frame (real RAM)
Tree: PML4 → PDPT → PD → PT, 512 entries each. TLB = cache of translations.

3 ArchMemory (= `process->getAddressSpace()`)

`mapPage(vpn, ppn, 1)`	vpn → ppn, user access. **false** if already mapped. `[[nodiscard]]`!
`unmapPage(vpn)`	removes mapping, **frees the ppn itself**, no TLB flush
`resolveMapping(vpn)`	→ `ArchMemoryMapping` (what is mapped?)
`checkAddressValid(addr)`	≠ 0 if mapped
`ArchMemory::flushTlb()`	*static* – after every `unmapPage`
`ArchMemory::getIdentAddressOfPPN(ppn)`	*static* – kernel pointer into a frame

ArchMemoryMapping – *fields, no* `()`

`.page_size`	**0 = not mapped**, else 4096
`.page_ppn`	the frame it points to
`.pt_ppn .pd_ppn .pdpt_ppn .pml4_ppn`	page-table pages on the way

```
ArchMemoryMapping m = process->getAddressSpace().resolveMapping(vpn);
if (m.page_size == 0) { /* free slot */ }
```

4 PageManager (physical frames)

```
size_t ppn = PageManager::instance()->allocPPN(); // free + ZEROED
PageManager::instance()->freePPN(ppn);    // only if NOT mapped
PageManager::instance()->getNumFreePages();
PageManager::instance()->getTotalNumPages();
```

`allocPPN` = get a frame, `mapPage` = connect it. You need **both**, in this order. `allocPPN()` takes **no** vpn (its parameter is a page *size*).

5 Locks

```
{
  ScopeLock lock(process->getThreadLock()); // acquire ...
  // check + change in ONE critical section
}                                           // ... released at }
```

- `ScopeLock` releases on **every** `return` automatically.
- By hand: `acquire()` / `release()` – then release before **every** `return`.
- `Condition`: `wait()`, `signal()`, `broadcast()` – mutex must be held.
- `allocPPN` *before* the lock = shorter critical section (give back with `freePPN` on failure).
- Never allocate while holding a `SpinLock`.

`thread_lock_` protects threads, stack slots & page tables. The page fault handler holds it for stack pages but calls `loadPage` **without** it.

6 Threads & registers

`currentThread->getTID()`	thread ID
`process->getPID()`, `getPPID()`	process IDs
`currentThread->user_registers_->rip`	where user code continues
`->rsp`	user stack pointer
`->rdi`, `->rsi`, `->rdx`	1st, 2nd, 3rd argument
`->rax`	return value
`process->createThread(entry,false,0,0)`	new `UserThread`
`process->findThreadDataByTid(tid)`	`UserThreadData*`
`process->exitProcess()`	kills **all** threads (POSIX)

`UserThreadData`: `thread_`, `exit_value_`, `tid_`, `exited_`, `joined_`, `requested_cancel_`, `detached_`
`UserThread`: `user_stack_top_`, `lowest_mapped_stack_vpn_`

7 Loader & page faults

- `common/source/mm/PageFaultHandler.cpp`: valid branch → stack growth (`isUserStackFault`, under the lock) → `getLoader().loadPage(address, getAddressSpace())` (no lock).
- New special regions (heap, magic page) go **first** in the valid branch, then `return`.
- **ELF**: sections (`.text .rodata .data .bss`) live in LOAD segments; `.bss` = MemSiz >

FileSiz (zeroed).

exit **666** no ELF segment for the address (`loadPage`)

exit **9999** invalid fault (null pointer, kernel address)

8 New syscall: the six places

1	`syscall-definitions.h`	`#define sc_foo 1505`
2	`Syscall.h`	`static size_t foo(size_t arg);`
3	`Syscall.cpp` switch	`case sc_foo: return_value = foo(arg1); break;`
4	`Syscall.cpp`	`size_t Syscall::foo(size_t arg){...}`
5	`userspace/libc/include/nonstd.h`	`int foo(void* p);`
6	`userspace/libc/src/nonstd.c`	`__syscall(sc_foo, (size_t)p, 0,0,0,0)`

Kernel headers: `common/include/kernel/`. Your block: **1500–1999** (taken: 1500 mkdir, 1501 join, 1502 cancel, 1503 testcancel, 1504 detach, 1998/1999). Also taken: 1000–1002 pthread_create/exit/self, 2000–2002 fork/getpid/getppid, 2500 execv. Free: 1505, 1510/1511, ... Then a test `userspace/tests/foo_basic.c`.

9 User pointers & return values

```
// ALWAYS before the kernel touches user memory (no overflow!):
if (buffer == 0 || size > USER_BREAK || buffer > USER_BREAK - size)
  return (size_t) -1;
Syscall::copyUserString(user_ptr, kernel_buf, buf_size); // strings
```

error	`return (size_t) -1;`	× `-1U` = only `0xFFFFFFFF`
NULL	`return 0;`	× `NULL` / `-1` in a `size_t` function
types	`size_t`	for addresses & sizes, never `int`

10 Debug output

```
debug(SYSCALL, "vpn %zx ppn %zx\n", vpn, ppn); // %zx hex, %zd dec
currentThread->printBacktrace();               // who called whom
// own flag in common/include/console/debug.h:
const size_t NOTE = Ansi_Cyan | OUTPUT_ENABLED;
```

Flags: `SYSCALL PAGEFAULT LOADER USERPROCESS USERTHREAD`

11 Shell

```
task start a0-02   # new task branch (training copy)
task test          # build + run the test
task run           # just run SWEB
task debug         # QEMU waits; CLion: 127.0.0.1:1234,
                   #   symbols /tmp/sweb-ag/kernel64.x
task diff a0-02    # compare with the solution
# interview = team repo, plain SWEB:
mkdir -p /tmp/sweb && cd /tmp/sweb
cmake ~/Documents/University/2026-fall/os/repos/osw26e3
make && make qemu
```

12 Pattern: find a free slot + map it (A0-02)

```
size_t Syscall::allocPage()
{
  UserProcess* process = currentThread->getProcess();
  ScopeLock lock(process->getThreadLock());
  for (size_t i = 0; i < ALLOC_REGION_PAGES; i++)
  {
    size_t vpn = ALLOC_REGION_START / PAGE_SIZE + i;
    if (process->getAddressSpace().resolveMapping(vpn).page_size == 0)
    {
      size_t ppn = PageManager::instance()->allocPPN();
      bool mapped = process->getAddressSpace().mapPage(vpn, ppn, 1);
      assert(mapped);
      return vpn * PAGE_SIZE;            // the ADDRESS, not the ppn
    }
  }
  return 0;                              // full -> NULL
}
// freePage: check aligned + in region + mapped (else -1),
//   unmapPage(vpn); ArchMemory::flushTlb(); return 0;   no freePPN!
```

13 Classic traps

1. `case` without `break`
2. `mapPage` result ignored (`[[nodiscard]]`)
3. `freePPN` after `unmapPage` → double free
4. no `flushTlb()` after `unmapPage`
5. check and map in two lock sections → race
6. lock still held at a `return` (manual acquire)
7. `vpn + i * PAGE_SIZE` instead of `vpn + i`
8. `-1` / `-1U` / `NULL` instead of `(size_t)-1` / `0`
9. `m.page_size()` – it's a field: `m.page_size`
10. `int` for addresses or sizes
11. `allocPPN` before checking the slot → leak

14 Tutor questions – say it out loud

- What does your lock protect? Show the critical section.
- Who frees the frame / the page-table pages? (`unmapPage`)
- Why flush the TLB? Would the test notice?
- What if `mapPage` returns false?
- What happens to pages never freed at exit? (`~UserProcess` deletes the address space)
- Why `size_t` and `(size_t)-1`? Why the overflow-safe pointer check?

15 Not every task is a syscall (A0-13 atexit)

Needs nothing only the kernel has? → only the libc stub:

```
// userspace/libc/src/stdlib.c              (exit() is in exec.c)
static void (*handlers[32])(void);  static int count = 0;
int atexit(void (*f)(void))
{ if (f == 0 || count == 32) return -1;  handlers[count++] = f;  return 0; }
void __runAtexitFunctions(void)          // exit() calls it before sc_exit
{ while (count > 0) { count--; handlers[count](); } }   // reverse, pop first
```

Missing: `strcpy/strncpy/strcat` (A0-14). Stubs: `malloc`, `sem_*`, `pthread_mutex/spin/cond_*`, `sleep`, `waitpid`, `mmap` (`ATLAS.md` §5).